Robot Operating System

Date - 17/03/2026 Anshita Menghnani

What is ROS 2?

The Robot Operating System (ROS) is a set of software libraries and tools that help in building robot applications. It offers a stable, secure, and production-ready environment for robotic applications, featuring improved real-time capabilities and performance.

It helps you:

- Break a robot system into small programs (nodes)
- Make them communicate with each other
- Handle real-time data, sensors, and control

Difference between ROS1 and ROS2:

	ROS 1	ROS 2
Middleware	TCPROS	DDS (Data Distribution Service)
Network Communication	Less robust	Quality of Service (QoS), Multicast
Real-time Support	Not officially supported	Full support with RTOS
Security	Basic or nonexistent	DDS Security
Software Compatibility	Linux only	Linux, Windows, Mac, RTOS
Scalability	Limited	High, ideal for fleets

Key differences between ROS1 AND ROS2:

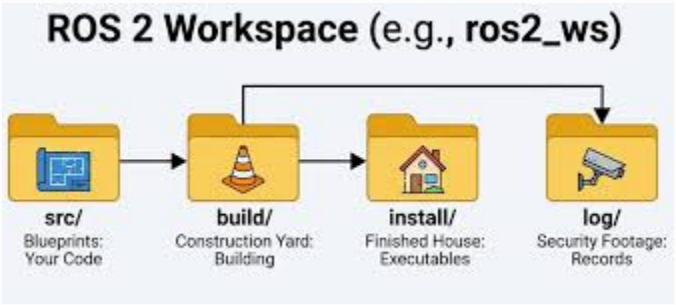
➔ **Architecture:** ROS 1 uses a centralized ROS Master for node discovery. ROS 2 is decentralized; nodes discover each other automatically via Data Distribution Service (DDS).

- ➔ **Performance/Real-time:** ROS 2 supports Quality of Service (QoS) policies (e.g., reliability, durability) and is designed for real-time applications.
- ➔ **Operating Systems:** ROS 1 is primarily Linux-based, whereas ROS 2 is built for full Windows, Linux, and MacOS compatibility.
- ➔ **Security:** ROS 2 includes security features (SROS2) by default, while ROS 1 lacked built-in security.
- ➔ **Language & Build:** ROS 2 officially supports C++17 and Python 3, with a new build tool (colcon)

What is a ROS 2 workspace?

A workspace is a directory containing ROS 2 packages. Before using ROS 2, it's necessary to source your ROS 2 installation workspace in the terminal you plan to work in. This makes ROS 2's packages available for you to use in that terminal. It is used for developing, building, and running robot applications.

Directory structure and build process using colcon:



1. src/
   - Contains all your ROS 2 packages (code)
   - You write/edit code here

2. build/
   - Stores temporary build files
   - Created automatically when you build

3. install/
   - Contains final usable files

- Used when running your nodes

4. log/
- Stores build logs and errors

In ROS 2, **colcon** is the command-line build tool used to compile and manage packages within a workspace. It supports multiple packages efficiently.

During the build process, three main directories are generated:

- The **build** directory contains intermediate compilation files.
- The **install** directory stores the final built packages and executables.
- The **log** directory keeps records of build outputs and error messages.

```
Write code → src/
      ↓
colcon build
      ↓
Files go to build/ + install/
      ↓
source install/setup.bash
      ↓
Run your nodes
```

What are ROS 2 nodes and how do these nodes communicate with each other?

A ROS 2 node is a small program that performs a specific task in a robotic system. Each node performs only one task. Nodes are easy to debug, update, and reuse.

ROS 2 nodes communicate via a decentralized peer-to-peer network using Data Distribution Service (DDS), requiring no central master. Nodes on the same or different machines (sharing the same ROS_DOMAIN_ID) automatically discover each other via multicast and exchange messages using Topics, Services, or Actions.

- **Topics** (continuous data flow)
- **Services** (request–response)
- **Actions** (long tasks with feedback)

What are ROS 2 topics?

A ROS 2 topic is a named channel used to send data between nodes continuously.

ROS 2 topics are named buses used by modular nodes to exchange data through a publish/subscribe mechanism, serving as a primary, one-way communication channel for continuous data streams like sensor feeds or motor commands. They are strongly-typed, allowing multiple nodes to publish or subscribe to data anonymously.

Publisher–Subscriber Model

This is how nodes communicate using topics (they don't communicate directly they do so through topics):

1. Publisher

- A node that sends data to a topic

2. Subscriber

- A node that receives data from that topic

What are ROS 2 messages?

ROS 2 messages are data structures used to send information between nodes. It tells us the format of data that is being transferred between different nodes.

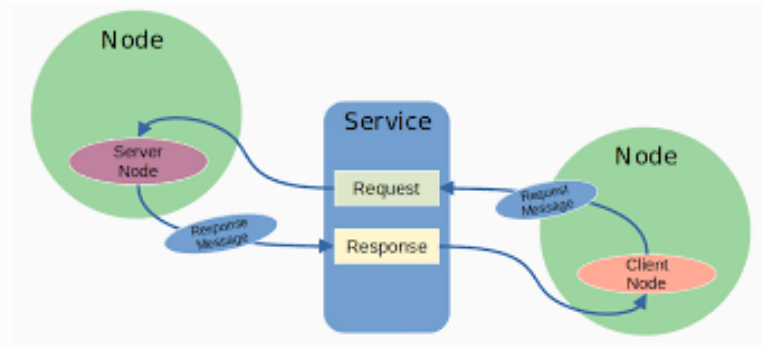
How to create custom messages?

Custom messages are created by defining fields in a .msg file and building them using colcon.

Custom messages in ROS 2 are created by defining a .msg file inside the package's msg folder, specifying the required data fields and types. The package configuration files (package.xml and CMakeLists.txt) are updated to support message generation, after which the workspace is built using colcon build and sourced.

What are ROS 2 services?

ROS 2 services are a communication method based on request–response. Client node sends request and server node sends back a response.



Servers vs topics

Communication type:
Topics = continuous data flow, Services = request–response

Direction:
Topics = one-way, Services = two-way

Usage:
Topics = sensor data streaming, Services = specific tasks (e.g., get status)

Timing:
Topics = ongoing, Services = only when requested

What is ROS 2 launch system?

The ROS 2 launch system is used to start and manage multiple nodes at once using a single command. It is used to start a complete robot system in one step.

ROS 2 run is used to run a single node. Syntax is as follows:

```
ros 2 run <package_name ><executable_name>
```

ROS2 launch is used to run multiple nodes using a launch file.

```
ros2 launch <package_name> <launch_file>
```

How does ROS 2 handle node discovery without a central master (roscore)?

In ROS 2, nodes discover each other using DDS (Data Distribution Service), so there is no need for a central master like roscore.

Peer to peer discovery:

- Each node automatically **announces its presence** on the network
- Other nodes **listen and detect** available nodes
- They directly connect and start communicating

Quality of Service (QoS) in ROS 2 and why it is important for sensor data.

Quality of Service (QoS) defines how data is sent and received between nodes. QoS in ROS 2 defines communication policies like reliability and durability, allowing optimization of data transfer, which is crucial for handling high-speed sensor data efficiently.

Why QoS is Important for Sensor Data

Sensor data (like camera or LiDAR) is:

- Very fast
- Continuous
- Sometimes okay to lose a few messages

Real world application of ROS 2

Autonomous Vehicles (Self-Driving Cars)

A widely used real-world application of ROS 2 is in autonomous vehicles, such as those developed by companies like Tesla. Multiple nodes process sensor data, plan paths, and control movement in real time, ensuring safe and reliable self-driving operation.

How ROS 2 is Used

- **Sensor Nodes** → collect data from cameras, radar, and LiDAR
- **Perception Nodes** → identify lanes, vehicles, pedestrians
- **Localization Nodes** → determine the car's position
- **Planning Nodes** → decide speed and path
- **Control Nodes** → steer, accelerate, and brake